

Principles of Programming Languages Semantics. An evaluator for IMP.

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Semantics: introduction

An evaluator for IMP

Denotational Semantics

Semantics

- ▶ Semantics is concerned with the **meaning** of language constructs
- ▶ Semantics must be **unambiguous**
- ▶ Semantics must be **flexible**

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Formal Semantics

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: Small-step SOS, Big-Step SOS

Formal Semantics

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: Small-step SOS, Big-Step SOS

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ **variables**
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= ' n) do  
    sum ::= sum + ' i;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10 ;  
i ::= 1 ;  
sum ::= 0 ;  
while (i <= n) do  
    sum ::= sum + i;  
    i ::= i + 1  
end
```

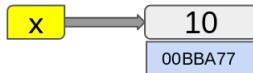
Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

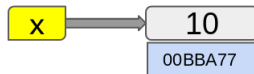
variables

- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



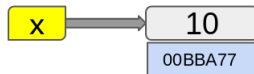
variables

- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



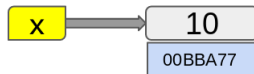
variables

- ▶ variable = *storage location at address* + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ *var_name* $\mapsto$ *value*
- ▶ Environment + memory



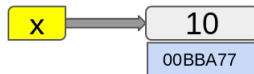
variables

- ▶ variable = *storage location* at *address* + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ *var_name* $\mapsto$ *value*
- ▶ *Environment + memory*



variables

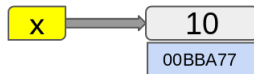
- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ *var_name* $\mapsto$ *value*
- ▶ Environment + memory



variables

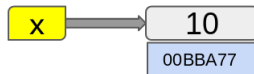
- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ $var_name \mapsto value$

▶ Environment + memory



variables

- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



Environment

- ▶ Environment: variables mapped to values

Definition `Env := string -> nat.`

Definition `env (string : string) :=
if (string_dec string "n")
then 10
else 0.`

- ▶ The environment is just a function

Compute `(env n) .`

`= 10`

`: nat`

Compute `(env i) .`

`= 0`

`: nat`

Check `env.`

`env : string -> nat`

Environment

- Environment: variables mapped to values

Definition `Env := string -> nat.`

Definition `env (string : string) :=
if (string_dec string "n")
then 10
else 0.`

- The environment is just a function

Compute `(env n) .`

`= 10`

`: nat`

Compute `(env i) .`

`= 0`

`: nat`

Check `env.`

`env : string -> nat`

Environment update

- Update = a new function s.t. the value for x is updated to v :

```
Definition update (x : string)
                  (v : nat)
                  (env : Env) : Env :=
fun y => if (string_dec x y)
then v
else env y.
```

Arithmetic expressions

► Syntax:

BNF:
AExp ::= $\mathbb{N}$
 | AExp '+' AExp
 | AExp '**' AExp

```
Inductive AExp :=  
  | anum : nat -> AExp  
  | aplus : AExp -> AExp -> AExp  
  | amul : AExp -> AExp -> AExp.
```

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
  | anum v => v  
  | aplus a1 a2 => (aeval a1) + (aeval a2)  
  | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```


Arithmetic expressions

► Syntax:

BNF:		Inductive AExp :=
AExp	::=	$\mathbb{N}$ anum : nat -> AExp
		AExp '+' AExp aplus : AExp -> AExp -> AExp
		AExp '*' AExp amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
    | anum v => v  
    | aplus a1 a2 => (aeval a1) + (aeval a2)  
    | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

Arithmetic expressions

► Syntax:

BNF:		Inductive AExp :=
AExp	::=	$\mathbb{N}$ anum : nat -> AExp
		AExp '+' AExp aplus : AExp -> AExp -> AExp
		AExp '*' AExp amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
    | anum v => v  
    | aplus a1 a2 => (aeval a1) + (aeval a2)  
    | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

Arithmetic expressions with *variables*

► Syntax:

BNF:
AExp ::= string
 | $\mathbb{N}$
 | AExp '+' AExp
 | AExp '**' AExp

```
Inductive AExp :=  
  | var : string -> AExp  
  | anum : nat -> AExp  
  | aplus : AExp -> AExp -> AExp  
  | amul : AExp -> AExp -> AExp.
```

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
  | var v => ???  
  | anum v => v  
  | aplus a1 a2 => (aeval a1) + (aeval a2)  
  | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

Arithmetic expressions with *variables*

► Syntax:

BNF:		Inductive AExp :=	
AExp	::=	string	var : string -> AExp
		N	anum : nat -> AExp
		AExp '+' AExp	aplus : AExp -> AExp -> AExp
		AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
  | var v => ???  
  | anum v => v  
  | aplus a1 a2 => (aeval a1) + (aeval a2)  
  | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

Arithmetic expressions with *variables*

► Syntax:

BNF:		Inductive AExp :=	
AExp	::=	string	var : string -> AExp
		$\mathbb{N}$	anum : nat -> AExp
		AExp '+' AExp	aplus : AExp -> AExp -> AExp
		AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
    | var v => ???  
    | anum v => v  
    | aplus a1 a2 => (aeval a1) + (aeval a2)  
    | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

Evaluate expression with variables

► Semantics:

```
Fixpoint aeval (a : AExp) (env : Env) : nat :=  
  match a with  
    | var v => env v  
    | anum v => v  
    | aplus a1 a2 => (aeval a1 env) + (aeval a2 env)  
    | amul a1 a2 => (aeval a1 env) * (aeval a2 env)  
  end.
```

► Notations and testing:

```
Coercion var : string >-> AExp.
```

```
Coercion anum : nat >-> AExp.
```

```
Notation "A + ' B" := (aplus A B) (at level 50).
```

```
Notation "A * ' B" := (amul A B) (at level 40).
```

```
Compute aeval (2 + ' 3 * ' 4) env.
```

```
= 14
```

```
: nat
```

```
Compute env n.
```

```
= 10
```

```
: nat
```

```
Compute aeval (2 + ' 3 * ' n) env.
```

```
= 32
```

```
: nat
```

Boolean expressions

► Syntax:

```
Inductive BExp :=  
  | btrue  : BExp  
  | bfalse : BExp  
  | blessthan : AExp -> AExp -> BExp  
  | band : BExp -> BExp -> BExp  
  | bnot : BExp -> BExp.
```

► Semantics:

```
Fixpoint beval (b : BExp) (env : Env) : bool :=  
  match b with  
    | btrue => true  
    | bfalse => false  
    | blessthan a1 a2 => Nat.leb (aeval a1 env) (aeval a2 env)  
    | band b1 b2 => andb (beval b1 env) (beval b2 env)  
    | bnot b' => negb (beval b' env)  
  end.
```

Boolean expressions

► Syntax:

```
Inductive BExp :=  
  | btrue  : BExp  
  | bfalse : BExp  
  | blessthan : AExp -> AExp -> BExp  
  | band : BExp -> BExp -> BExp  
  | bnot : BExp -> BExp.
```

► Semantics:

```
Fixpoint beval (b : BExp) (env : Env) : bool :=  
  match b with  
    | btrue => true  
    | bfalse => false  
    | blessthan a1 a2 => Nat.leb (aeval a1 env) (aeval a2 env)  
    | band b1 b2 => andb (beval b1 env) (beval b2 env)  
    | bnot b' => negb (beval b' env)  
  end.
```


Assignments

► Syntax:

```
Inductive Stmt :=  
  | assignment : string -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
: Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval(s : Stmt) (env : Env) : Env :=  
match s with  
  | assignment x a => update x (aeval a env) env  
end.
```

Assignments

► Syntax:

Inductive Stmt :=

| assignment : string -> AExp -> Stmt.

Notation "A ::= B" := (assignment A B) (at level 54).

Check n ::= 100 .

n ::= 100

: Stmt

► Assignments modify the environment

► Semantics:

Fixpoint eval(s : Stmt) (env : Env) : Env :=

match s **with**

| assignment x a => update x (aeval a env) env

end.

Assignments

► Syntax:

```
Inductive Stmt :=  
  | assignment : string -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
: Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
  | assignment x a => update x (aeval a env) env  
end.
```

Programs: sequence of statements

► Syntax:

Inductive Stmt :=

| seq s1 s2 => eval s2 (eval s1 env)

Notation "S S'" := (assignment S S') (at level 54).

Check n ::= 100 ;; i ::= 7 .

n ::= 100 ;; x i ::= 7

: Stmt

► Semantics:

Fixpoint eval (s : Stmt) (env : Env) : Env :=

 match s with

 | seq s1 s2 => eval s2 (eval s1 env)

 end.

Programs: sequence of statements

► Syntax:

Inductive Stmt :=

| seq s1 s2 => eval s2 (eval s1 env)

Notation "S S'" := (assignment S S') (at level 54).

Check n ::= 100 ;; i ::= 7 .

n ::= 100 ;; x i ::= 7

: Stmt

► Semantics:

Fixpoint eval (s : Stmt) (env : Env) : Env :=

match s **with**

 | seq s1 s2 => eval s2 (eval s1 env)

end.

Loops

- Syntax:

```
Inductive Stmt :=  
| while : BExp -> Stmt -> Stmt.
```

- Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
  match s with  
  | while b s' => if (beval b env)  
    then (eval (seq s' (while b s')) env)  
    else env  
  end.
```

- ERROR:

Error: Cannot guess decreasing argument of fix.

- (Partial) solution: add a *fuel* parameter

Loops

- Syntax:

```
Inductive Stmt :=  
  | while : BExp -> Stmt -> Stmt.
```

- Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
  match s with  
    | while b s' => if (beval b env)  
      then (eval (seq s' (while b s')) env)  
      else env  
  end.
```

- ERROR:

Error: Cannot guess decreasing argument of fix.

- (Partial) solution: add a *fuel* parameter

Loops

- Syntax:

```
Inductive Stmt :=  
  | while : BExp -> Stmt -> Stmt.
```

- Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
  match s with  
    | while b s' => if (beval b env)  
      then (eval (seq s' (while b s')) env)  
      else env  
  end.
```

- ERROR:

Error: Cannot guess decreasing argument of fix.

- (Partial) solution: add a *fuel* parameter

Loops

- Syntax:

```
Inductive Stmt :=  
  | while : BExp -> Stmt -> Stmt.
```

- Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
  match s with  
    | while b s' => if (beval b env)  
      then (eval (seq s' (while b s')) env)  
      else env  
  end.
```

- ERROR:

Error: Cannot guess decreasing argument of fix.

- (Partial) solution: add a *fuel* parameter

Compositionality

- ▶ The evaluation of expressions is *compositional*: *the evaluation of an expression is defined **purely** in terms of the evaluation of its subexpressions.*
- ▶ In our case, the evaluation of loops breaks compositionality: the evaluation of `(while b s')` is defined as the evaluation of `(seq s' (while b s'))`.
- ▶ **Fixpoint** `eval (s : Stmt) (env : Env) : Env :=`
 `match s with`
 `| while b s' => if (beval b env)`
 `then (eval (seq s' (while b s')) env)`
 `else env`

 `end.`
- ▶ A possible solution is to define an operational semantics (using relations instead of functions) → will be discussed in a future lecture.

Compositionality

- ▶ The evaluation of expressions is *compositional*: the evaluation of an expression is defined **purely** in terms of the evaluation of its subexpressions.
- ▶ In our case, the evaluation of loops breaks compositionality: the evaluation of `(while b s')` is defined as the evaluation of `(seq s' (while b s'))`.
- ▶ **Fixpoint** `eval (s : Stmt) (env : Env) : Env :=`
 `match s with`
 `| while b s' => if (beval b env)`
 `then (eval (seq s' (while b s')) env)`
 `else env`
 `end.`
- ▶ A possible solution is to define an operational semantics (using relations instead of functions) → will be discussed in a future lecture.

Denotational Semantics

- ▶ Denotational Semantics is an approach to define the meaning of programs.
- ▶ The basic idea is to use mathematical functions to specify what programs mean.
- ▶ Valuation function: $\llbracket \cdot \rrbracket. : AExp \times Env \rightarrow Int$

The denotation of arithmetic expressions in an environment σ is defined as:

$$\llbracket \text{var } x \rrbracket_{\sigma} = \sigma(x)$$

$$\llbracket \text{num } n \rrbracket_{\sigma} = n$$

$$\llbracket \text{aplus } a_1 \ a_2 \rrbracket_{\sigma} = \llbracket a_1 \rrbracket_{\sigma} + \llbracket a_2 \rrbracket_{\sigma} \quad \llbracket \text{amul } a_1 \ a_2 \rrbracket_{\sigma} = \llbracket a_1 \rrbracket_{\sigma} * \llbracket a_2 \rrbracket_{\sigma}$$

Boolean expressions

- Valuation function: $\llbracket \cdot \rrbracket. : BExp \times Env \rightarrow Bool$

The denotation of boolean expressions in an environment σ is defined as:

$$\begin{array}{ll} \llbracket btrue \rrbracket_{\sigma} = & true \\ \llbracket bfalse \rrbracket_{\sigma} = & false \\ \llbracket blessthan \mathbf{a_1} \mathbf{a_2} \rrbracket_{\sigma} = & \llbracket \mathbf{a_1} \rrbracket_{\sigma} \leq \llbracket \mathbf{a_2} \rrbracket_{\sigma} \\ \llbracket bnot \mathbf{b} \rrbracket_{\sigma} = & \neg \llbracket \mathbf{b} \rrbracket_{\sigma} \\ \llbracket band \mathbf{b_1} \mathbf{b_2} \rrbracket_{\sigma} = & \llbracket \mathbf{b_1} \rrbracket_{\sigma} \wedge \llbracket \mathbf{b_2} \rrbracket_{\sigma} \end{array}$$

Statements

- Valuation function: $\langle \cdot \rangle \cdot : Stmt \rightarrow (Env \rightarrow Env)$

The denotation of statements in an environment σ is defined as:

$$\begin{aligned}\langle \text{assignment } x \ a \rangle_{\sigma} &= \text{update } x \llbracket a \rrbracket_{\sigma} \sigma \\ \langle \text{seq } s_1 \ s_2 \rangle_{\sigma} &= \langle s_2 \rangle_{\langle s_1 \rangle_{\sigma}} \\ \langle \text{skip} \rangle_{\sigma} &= \sigma \\ \langle \text{ite } b \ s_1 \ s_2 \rangle_{\sigma} &= \begin{cases} \langle s_1 \rangle_{\sigma}, & \text{if } \llbracket b \rrbracket_{\sigma} \\ \langle s_2 \rangle_{\sigma}, & \text{if } \neg \llbracket b \rrbracket_{\sigma} \end{cases} \\ \langle \text{while } b \text{ do } s \rangle_{\sigma} &= \langle \text{ite } b \ (\text{seq } s \ (\text{while } b \text{ do } s)) \ \text{skip} \rangle_{\sigma}\end{aligned}$$

Compositionality issue

- Valuation function: $\langle \cdot \rangle \cdot : Stmt \rightarrow (Env \rightarrow Env)$

The denotation of statements in an environment σ is defined as:

$$\begin{aligned}\langle \text{assignment } x \ a \rangle_{\sigma} &= \text{update } x \llbracket a \rrbracket_{\sigma} \sigma \\ \langle \text{seq } s_1 \ s_2 \rangle_{\sigma} &= \langle s_2 \rangle_{\langle s_1 \rangle_{\sigma}} \\ \langle \text{skip} \rangle_{\sigma} &= \sigma \\ \langle \text{ite } b \ s_1 \ s_2 \rangle_{\sigma} &= \begin{cases} \langle s_1 \rangle_{\sigma}, & \text{if } \llbracket b \rrbracket_{\sigma} \\ \langle s_2 \rangle_{\sigma}, & \text{if } \neg \llbracket b \rrbracket_{\sigma} \end{cases} \\ \langle \text{while } b \text{ do } s \rangle_{\sigma} &= \langle \text{ite } b \ (\text{seq } s \ (\text{while } b \text{ do } s)) \ \text{skip} \rangle_{\sigma}\end{aligned}$$

The definition for the while loop is not defined *purely* in terms of the denotations of its subcomponents.

Evaluation as a relation – in a future lecture

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation – in a future lecture

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation – in a future lecture

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation – in a future lecture

► Assignment:
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► Sequence:
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► Loop (true case):

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► Loop (false case):
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Bibliography

- ▶ Chapter Simple Imperative Programs in **Software Foundations - Volume 1**, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey
<https://softwarefoundations.cis.upenn.edu/lf-current/Imp.html>